

MINISTRY OF EDUCATION OF REPUBLIC OF MOLDOVA
TECHNICAL UNIVERSITY OF MOLDOVA
FACULTY OF COMPUTERS, INFORMATICS AND MICROELECTRONICS
SOFTWARE ENGINEERING DEPARTMENT

EMBEDDED SYSTEMS
LABORATORY WORK WORK #5.2

PID Control of a Servo-Driven Gate

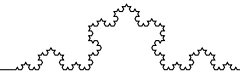
During the preparation of this report, the author used various AI Tools such as ChatGPT, Gemini, Claude to generate/augment the content, generate the code and structure the directory. The resulting information was reviewed, validated, and adjusted to meet the requirements of the laboratory assignment.

Author: Timur CRAVTOV
std. gr. FAF-231

Verified by:
Alexei MARTÎNIUC
asist. univ



Chişinău, 2026



1 Objective and Technical Analysis

The objective of this laboratory work is to implement closed-loop PID control for a servo-driven gate using an ultrasonic distance sensor as feedback. Compared with the previous lab, the hardware layout remains the same, while the control strategy is upgraded from open-loop or threshold-based regulation to proportional-integral-derivative compensation.

The system is organized into three cooperating tasks: input acquisition, PID control, and state reporting. This keeps timing deterministic and separates user interaction from the control loop.

The main learning outcomes are:

- understanding the structure of a PID controller and its discrete-time implementation;
- converting a distance error into a bounded servo command;
- applying integral windup protection and output clamping;
- keeping a FreeRTOS-based control loop responsive under serial and keypad interaction;
- documenting the same electrical scheme used in Lab 5.1.

1.1 PID Control Theory

The control signal is computed from the distance error between the desired threshold and the measured distance:

$$e[k] = r[k] - y[k] \quad (1)$$

where $r[k]$ is the setpoint and $y[k]$ is the measured distance.

The discrete PID law used in the implementation is:

$$u[k] = K_p e[k] + K_i \sum_{i=0}^k e[i] \Delta t + K_d \frac{e[k] - e[k-1]}{\Delta t} \quad (2)$$

The output is then limited to the supported servo position range:

$$u_{sat}[k] = \min(\max(u[k], u_{min}), u_{max}) \quad (3)$$

This is important because the actuator should never be commanded outside its safe travel interval.



1.2 Control Objective

The controller keeps the servo closed when the object is out of range and opens the gate proportionally when the measured distance moves inside the configured threshold. That behavior gives the system a clear binary default state while still allowing smooth intermediate motion when the error is small.

2 System Design

2.1 Task-Based Architecture

The software is split into three tasks with fixed periods:

- **TaskRead** (40 ms): reads keypad input and updates the distance threshold;
- **TaskFilter** (80 ms): reads the ultrasonic sensor and executes PID control;
- **TaskDisplay** (250 ms): reports the current state over serial and Teleplot output.

This structure keeps the control calculation periodic and prevents input handling from delaying servo updates.

2.2 Control Flow

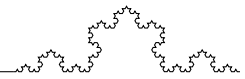
PID control loop: keypad threshold $\rightarrow$ distance measurement $\rightarrow$ error
calculation $\rightarrow$ PID update $\rightarrow$ bounded servo command

Figure 1: PID Control Loop for Lab 5.2

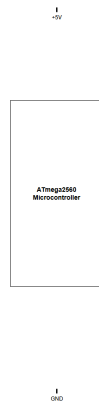
The controller follows the same flow on every cycle. If the target is outside the valid range, the actuator is returned to the fully closed position and the integral state is cleared to avoid stale control memory.

2.3 Electrical Schematic

The electrical scheme is intentionally kept the same as in Lab 5.1. The hardware topology uses the same Arduino Nano-based layout with the servo actuator, keypad, LCD, and ultrasonic sensor connected to the same logical blocks.



Lab 4.1 - Binary Actuator Control Circuit Schematic



Signal Conditioning Pipeline



(a) Electrical schematic (reused from Lab 5.1)

(b) Signal conditioning and PID flow

Figure 2: Hardware schematic and signal-flow diagrams. Export the PNGs from the provided ‘.drawio‘ sources in ‘Images/work‘ using your Draw.io CLI.

The report keeps the same scheme because the assignment changes the control law, not the physical I/O arrangement. Only the software logic is extended to compute a PID command from sensor feedback.

2.4 Pin Assignment Summary

Table 1: Main Hardware Connections

Component	Pin(s)	Function
Servo actuator	D11	PWM output
Keypad rows	D9, D8, D7, D6	Input scanning
Keypad columns	D5, D4, D3, D2	Input scanning
LCD I2C bus	A4, A5	Display interface
Ultrasonic sensor	A0, A2	Distance feedback

3 Implementation and Results

3.1 Software Structure

The implementation uses shared state variables for measured distance, current servo angle, PID error, and output position. The control task updates these fields atomically and the display task reads them for reporting.

The main controller logic can be summarized as follows:

```

1 if (valid && distanceCm <= minDistanceCm) {
2     float error = setpoint - distanceCm;
3     pidIntegral = clampFloat(pidIntegral + error * dt,
4                             PID_INTEGRAL_MIN, PID_INTEGRAL_MAX);

```



```

5   float derivative = (error - lastError) / dt;
6   float control = PID_KP * error + PID_KI * pidIntegral + PID_KD * derivative;
7   int16_t command = clampInt16(SERVO_POS_MIN + roundToInt(control),
8                               SERVO_POS_MIN, SERVO_POS_MAX);
9   actuator->setPosition((uint8_t)command);
10 }

```

This structure shows the core PID computation, the integral limiter, and the final output saturation.

3.2 Timing Characteristics

Table 2: Control Loop Timing

Task	Period
TaskRead	40 ms
TaskFilter	80 ms
TaskDisplay	250 ms
Teleplot output	80 ms

The shorter control period keeps the servo responsive while the display task remains slower to reduce serial noise and processing overhead.

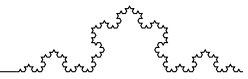
3.3 Observed Behavior

The controller produces smooth servo movement when the sensed distance crosses the active threshold. For stable targets, the proportional term dominates; for slowly drifting targets, the integral term removes steady-state error; and the derivative term damps sudden changes.

When the object leaves the control zone, the actuator returns to the closed position and the PID memory is reset. This prevents accumulated integral error from causing an overshoot when the target reappears.

3.4 Practical Notes

- The servo command is intentionally bounded to the safe travel range.
- Integral windup protection is required because the output may stay saturated for extended periods.
- The ultrasonic sensor reading is treated as invalid outside the configured physical range.
- The keypad provides a convenient way to adjust the threshold without recompiling firmware.



4 Conclusion

This laboratory work implemented a discrete PID controller for a servo-driven gate and kept the same hardware layout as Lab 5.1. The result is a stable embedded control system with bounded actuator output, clear task separation, and simple operator input through the keypad.

The main achievement is the transition from fixed-position logic to feedback-based regulation. Compared with threshold-only control, the PID approach improves smoothness and reduces oscillation near the target distance.

Future improvements could include automatic gain tuning, sensor filtering for noisier environments, and a richer LCD user interface for live parameter adjustment.

References

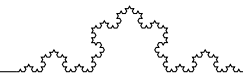
- [1] K. J. Astrom and T. Hagglund, *PID Controllers: Theory, Design, and Tuning*, 2nd ed., Instrument Society of America, 1995.
- [2] FreeRTOS API Reference, <https://www.freertos.org/a00106.html>
- [3] Arduino Servo Library Reference, <https://www.arduino.cc/reference/en/libraries/servo/>
- [4] HC-SR04 Ultrasonic Sensor Datasheet, generic module reference.

A Source Code

Full source code for Lab 5.2 is available in the repository.

`actuator.h`

```
1 #pragma once
2
3 #include <Arduino.h>
4 #include <Servo.h>
5
6 // Configuration constants
7 constexpr uint8_t ACTUATOR_MIN = 0;
8 constexpr uint8_t ACTUATOR_MAX = 100;
9
10 // Abstract interface for analog actuators
11 class AnalogActuator {
12     public:
```

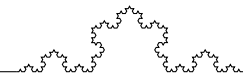


```
13     virtual void setPosition(uint8_t position) = 0;
14     virtual uint8_t getPosition() = 0;
15     virtual void begin() = 0;
16     virtual ~AnalogActuator() {}
17 };
18
19 // Concrete implementation for Servo-based actuator control
20 // Maps 0–100% to servo angle (0–180 degrees)
21 class ServoActuator : public AnalogActuator {
22     private:
23         uint8_t pin;
24         Servo servo;
25         uint8_t currentPosition; // 0–100%
26         uint8_t currentAngle;   // 0–180 degrees
27
28     public:
29         ServoActuator(uint8_t p);
30         void begin() override;
31         void setPosition(uint8_t position) override;
32         uint8_t getPosition() override;
33         uint8_t getAngle();
34 };
```

Listing 1: actuator.h - Actuator Interface

actuator.cpp

```
1 #include "actuator.h"
2
3 ServoActuator::ServoActuator(uint8_t p) : pin(p),
4     currentPosition(0), currentAngle(0) {}
5
6 void ServoActuator::begin() {
7     servo.attach(pin);
8     servo.write(0); // Start at 0 degrees
9 }
10
11 void ServoActuator::setPosition(uint8_t position) {
12     // Saturate to valid range
13     if (position > ACTUATOR_MAX) position = ACTUATOR_MAX;
```

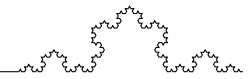


```
14     currentPosition = position;
15     // Map 0–100% to 0–180 degrees
16     currentAngle = map(position, ACTUATOR_MIN, ACTUATOR_MAX, 0,
17                        180);
18     servo.write(currentAngle);
19 }
20 uint8_t ServoActuator::getPosition() {
21     return currentPosition;
22 }
23
24 uint8_t ServoActuator::getAngle() {
25     return currentAngle;
26 }
```

Listing 2: actuator.cpp - Actuator Implementation

tasks.h

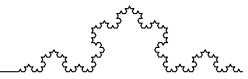
```
1 #ifndef TASKS_H
2 #define TASKS_H
3 #include "actuator.h"
4 #include <Arduino_FreeRTOS.h>
5 #include <sensors/distance.h>
6
7 // Task periods (ms)
8 constexpr uint32_t TASK_READ_PERIOD = 40;
9 constexpr uint32_t TASK_CONTROL_PERIOD = 80;
10 constexpr uint32_t TASK_DISPLAY_PERIOD = 250;
11 constexpr uint32_t TASK_PLOT_PERIOD = 80;
12
13 // Distance limits (cm)
14 constexpr uint16_t MIN_DISTANCE_DEFAULT_CM = 15;
15 constexpr uint16_t MIN_DISTANCE_MIN_CM = 2;
16 constexpr uint16_t MIN_DISTANCE_MAX_CM = 400;
17
18 // Servo positions (%)
19 constexpr uint8_t SERVO_POS_MIN = 0;
20 constexpr uint8_t SERVO_POS_MAX = 40;
21 constexpr uint8_t SERVO_POS_CLOSED = SERVO_POS_MIN;
22 constexpr uint8_t SERVO_POS_OPEN = SERVO_POS_MAX;
```

```

23 constexpr uint8_t SERVO_POS_NEUTRAL = (SERVO_POS_MIN +
    SERVO_POS_MAX) / 2;
24
25 // Servo angle helpers (0–180 degrees)
26 constexpr uint8_t SERVO_ANGLE_MIN      = (uint8_t)((SERVO_POS_MIN
    * 180u) / ACTUATOR_MAX);
27 constexpr uint8_t SERVO_ANGLE_MAX      = (uint8_t)((SERVO_POS_MAX
    * 180u) / ACTUATOR_MAX);
28 constexpr uint8_t SERVO_ANGLE_NEUTRAL = (uint8_t)((
    SERVO_POS_NEUTRAL * 180u) / ACTUATOR_MAX);
29 constexpr uint8_t SERVO_ANGLE_DEADBAND = 5;
30
31 // Keep a small hysteresis to avoid gate chatter near threshold
32 constexpr uint8_t GATE_HYSTERESIS_CM = 2;
33
34 // Serial Plotter output toggle (prints only plot lines when
    true)
35 constexpr bool ENABLE_PLOTTER = true;
36
37 // PID tuning (start values; adjust on hardware)
38 constexpr float PID_KP          = 1.2f;
39 constexpr float PID_KI          = 0.02f;
40 constexpr float PID_KD          = 0.8f;
41 constexpr float PID_INTEGRAL_MIN = -120.0f;
42 constexpr float PID_INTEGRAL_MAX = 120.0f;
43 constexpr bool  PID_INVERT_OUTPUT = false;
44
45 // Shared state variables (accessed by tasks)
46 extern volatile uint16_t measuredDistanceCm;
47 extern volatile uint16_t minDistanceCm;
48 extern volatile uint8_t  servoAngle;
49 extern volatile int16_t  pidErrorCm;
50 extern volatile uint8_t  pidOutputPos;
51 extern volatile bool     gateOpen;
52 extern volatile bool     sensorValid;
53 extern volatile char     lastKeyPressed;
54
55 // System state

```



```

56 struct SystemState {
57     uint16_t distanceCm;
58     uint16_t thresholdCm;
59     uint8_t angle;
60     int16_t errorCm;
61     uint8_t outputPos;
62     bool isOpen;
63     bool isSensorValid;
64     char key;
65 };
66 extern volatile SystemState sysState;
67
68 struct ControlContext {
69     ServoActuator* actuator;
70     DistanceSensor* sensor;
71 };
72
73 // Task function declarations
74 void TaskRead(void* pvParameters);
75 void TaskFilter(void* pvParameters);
76 void TaskDisplay(void* pvParameters);
77
78 #endif // TASKS_H

```

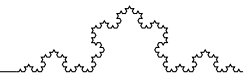
Listing 3: tasks.h - Task Interface

tasks.cpp

```

1 #include "tasks.h"
2 #include <Arduino.h>
3 #include <Arduino_FreeRTOS.h>
4 #include <avr/pgmspace.h>
5 #include <stdio.h>
6 #include <string.h>
7
8 // progmem strings
9 static const char STR_LINE0[]          PROGMEM = "\rD:%3u T:%3u %
    c  \n";
10 static const char STR_LINE0_INVALID[]  PROGMEM = "\rD:--- T:%3u %
    c  \n";
11 static const char STR_LINE1[]          PROGMEM = "\rK:%c In:%-3s

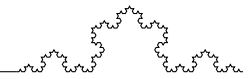
```



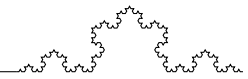
```

    ";
12
13 // plotter format strings
14 static const char PLOT_DIST[]    PROGMEM = ">pos/dist:%d.%u\n";
15 static const char PLOT_SETPOINT[] PROGMEM = ">pos/setpoint:%u\n"
    ;
16 static const char PLOT_ERROR[]    PROGMEM = ">pid/error:%d.%u\n"
    ;
17 static const char PLOT_OUTPUT[]    PROGMEM = ">pid/output:%d\n";
18 static const char PLOT_VALID[]    PROGMEM = ">status/valid:%u\n"
    ;
19
20 // shared state variables
21 volatile uint16_t    measuredDistanceCm = 0;
22 volatile uint16_t    minDistanceCm      =
    MIN_DISTANCE_DEFAULT_CM;
23 volatile uint8_t     servoAngle          = 0;
24 volatile int16_t     pidErrorCm          = 0;
25 volatile uint8_t     pidOutputPos        = SERVO_POS_MIN;
26 volatile bool        gateOpen            = false;
27 volatile bool        sensorValid         = false;
28 volatile char        lastKeyPressed      = '_';
29 volatile SystemState sysState            = {0,
    MIN_DISTANCE_DEFAULT_CM, 0, 0,
30                                     SERVO_POS_MIN, false,
                                     false, '_' };
31
32 // keypad input buffer
33 static char    inputBuffer[4] = {0};
34 static uint8_t inputIndex      = 0;
35
36 // resets the input buffer
37 static inline void clearInputBuffer() {
38     inputIndex      = 0;
39     inputBuffer[0] = '\0';
40 }
41
42 // converts the buffer to an integer

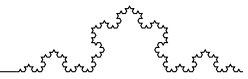
```



```
43 static uint16_t parseInputBuffer() {
44     uint16_t value = 0;
45     for (uint8_t i = 0; i < inputIndex; i++) {
46         value = (uint16_t)(value * 10u + (uint16_t)(inputBuffer[
47             i] - '0'));
48     }
49     return value;
50 }
51 // bounds the threshold value
52 static inline uint16_t clampThreshold(uint16_t value) {
53     if (value < MIN_DISTANCE_MIN_CM) return MIN_DISTANCE_MIN_CM;
54     if (value > MIN_DISTANCE_MAX_CM) return MIN_DISTANCE_MAX_CM;
55     return value;
56 }
57
58 // bounds a float value
59 static inline float clampFloat(float value, float minValue,
60     float maxValue) {
61     if (value < minValue) return minValue;
62     if (value > maxValue) return maxValue;
63     return value;
64 }
65 // bounds an int16 value
66 static inline int16_t clampInt16(int16_t value, int16_t minValue
67     , int16_t maxValue) {
68     if (value < minValue) return minValue;
69     if (value > maxValue) return maxValue;
70     return value;
71 }
72 // rounds a float to the nearest int16
73 static inline int16_t roundToInt(float value) {
74     return (int16_t)(value + ((value >= 0.0f) ? 0.5f : -0.5f));
75 }
76
77 // fixed point moving average shift
```



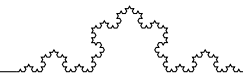
```
78 #define EMA_SHIFT 3
79
80 // calculates exponential moving average step
81 static inline int16_t emaStep(int16_t smoothed, int16_t incoming
82 ) {
83     return (int16_t)(smoothed + ((incoming - smoothed) >>
84         EMA_SHIFT));
85 }
86
87 // prints fixed point value with one decimal
88 static void printFixed(const char* fmt_P, int16_t val_x10) {
89     int16_t intPart;
90     uint8_t fracPart;
91     if (val_x10 < 0) {
92         int16_t abs_x10 = (int16_t)(-val_x10);
93         intPart = (int16_t)(-(abs_x10 / 10));
94         fracPart = (uint8_t) ( abs_x10 % 10u);
95     } else {
96         intPart = (int16_t)(val_x10 / 10);
97         fracPart = (uint8_t) (val_x10 % 10u);
98     }
99     printf_P(fmt_P, (int)intPart, (unsigned)fracPart);
100 }
101
102 // pid variables
103 static float pidIntegral = 0.0f;
104 static float lastError   = 0.0f;
105
106 // reads serial inputs for system configuration
107 void TaskRead(void* pvParameters) {
108     (void)pvParameters;
109     while (true) {
110         char c = 0;
111         if (scanf("%c", &c) > 0 && c != '\0' && c != '\n' && c
112             != '\r') {
113             lastKeyPressed = c;
114             sysState.key    = c;
115         }
116     }
117 }
```



```

113         if (c >= '0' && c <= '9') {
114             if (inputIndex < 3) {
115                 inputBuffer[inputIndex++] = c;
116                 inputBuffer[inputIndex] = '\0';
117             }
118         } else if (c == '#' || c == 'A') {
119             if (inputIndex > 0) {
120                 minDistanceCm = clampThreshold(
121                     parseInputBuffer());
122                 sysState.thresholdCm = minDistanceCm;
123                 clearInputBuffer();
124             }
125         } else if (c == '*' || c == 'B') {
126             clearInputBuffer();
127         } else if (c == 'C') {
128             minDistanceCm = MIN_DISTANCE_DEFAULT_CM;
129             sysState.thresholdCm = minDistanceCm;
130             clearInputBuffer();
131         } else if (c == 'D') {
132             minDistanceCm = clampThreshold((uint16_t)
133                 (minDistanceCm + 5u));
134             sysState.thresholdCm = minDistanceCm;
135             clearInputBuffer();
136         }
137     }
138 }
139
140 // executes sensor reading and pid control
141 void TaskFilter(void* pvParameters) {
142     ControlContext* ctx = static_cast<ControlContext*>(
143         pvParameters);
144     ServoActuator* actuator = ctx->actuator;
145     DistanceSensor* sensor = ctx->sensor;
146
147     // start fully closed
148     actuator->setPosition(SERVO_POS_MIN);

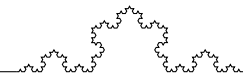
```



```

148     vTaskDelay(pdMS_TO_TICKS(300));
149     gateOpen    = false;
150     servoAngle = actuator->getAngle();
151
152     while (true) {
153         const float rawDistance = sensor->readRaw();
154         const bool  valid       = (rawDistance > 0.5f) &&
155                                   (rawDistance <= (float)
156                                                MIN_DISTANCE_MAX_CM);
157
158         const uint16_t distanceCm = valid ? (uint16_t)(
159             rawDistance + 0.5f) : 0u;
160
161         // calculates pid only when object is within threshold
162         if (valid && distanceCm <= minDistanceCm) {
163             const float setpoint = (float)minDistanceCm;
164             float error = setpoint - (float)distanceCm;
165
166             // zeroes error near threshold to stop micro-
167             // movements;
168             // integral is preserved so the servo can still
169             // reach SERVO_POS_MAX
170             if (error >= -1.0f && error <= 1.0f) {
171                 error = 0.0f;
172             }
173
174             if (PID_INVERT_OUTPUT) error = -error;
175
176             const float dt = (float)TASK_CONTROL_PERIOD / 1000.0
177                             f;
178             pidIntegral    = clampFloat(pidIntegral + (error *
179                 dt),
180
181                                     PID_INTEGRAL_MIN,
182                                     PID_INTEGRAL_MAX);
183
184             const float derivative = (error - lastError) / dt;
185             float control = (PID_KP * error)
186                             + (PID_KI * pidIntegral)
187                             + (PID_KD * derivative);

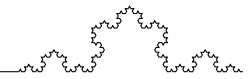
```



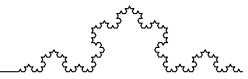
```

179         // zeroes out negligible control signals to keep
           servo still
180     if (control > -2.0f && control < 2.0f) {
181         control = 0.0f;
182     }
183
184     // command offset from SERVO_POS_MIN: zero control =
           fully closed,
185     // positive control opens toward SERVO_POS_MAX
186     int16_t command = (int16_t)SERVO_POS_MIN +
           roundToInt(control);
187     command = clampInt16(command, SERVO_POS_MIN,
           SERVO_POS_MAX);
188     actuator->setPosition((uint8_t)command);
189
190     gateOpen      = (command > SERVO_POS_MIN);
191     lastError      = error;
192     pidErrorCm     = roundToInt(error);
193     pidOutputPos   = (uint8_t)command;
194 } else {
195     // object out of range - drive fully closed, clear
           pid memory
196     actuator->setPosition(SERVO_POS_MIN);
197     gateOpen      = false;
198     pidIntegral    = 0.0f;
199     lastError      = 0.0f;
200     pidErrorCm     = 0;
201     pidOutputPos   = SERVO_POS_MIN;
202 }
203
204 measuredDistanceCm = distanceCm;
205 sensorValid        = valid;
206 servoAngle         = actuator->getAngle();
207
208 sysState.distanceCm = measuredDistanceCm;
209 sysState.thresholdCm = minDistanceCm;
210 sysState.angle       = servoAngle;
211 sysState.errorCm     = pidErrorCm;

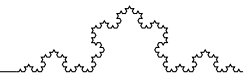
```

```
212     sysState.outputPos      = pidOutputPos;
213     sysState.isOpen         = gateOpen;
214     sysState.isSensorValid = sensorValid;
215
216     vTaskDelay(pdMS_TO_TICKS(TASK_CONTROLPERIOD));
217 }
218 }
219
220 // displays output on serial and processes telemetry
221 void TaskDisplay(void* pvParameters) {
222     (void)pvParameters;
223
224     // moving average variables
225     static int16_t smoothDist_x10 = 0;
226     static int16_t smoothErr_x10  = 0;
227     static int16_t smoothOut      = 0;
228     static bool    smoothReady    = false;
229
230     while (true) {
231         // reads shared state
232         uint16_t dLocal, tLocal;
233         uint8_t  angleLocal, outLocal;
234         int16_t  errLocal;
235         bool     validLocal;
236         char     keyLocal;
237         char     localInput[4] = "____";
238
239         taskENTER_CRITICAL();
240         dLocal      = sysState.distanceCm;
241         tLocal      = sysState.thresholdCm;
242         angleLocal  = sysState.angle;
243         errLocal    = sysState.errorCm;
244         outLocal    = sysState.outputPos;
245         validLocal  = sysState.isSensorValid;
246         keyLocal    = sysState.key;
247         if (inputIndex > 0) {
248             strncpy(localInput, inputBuffer, 3);
249             localInput[3] = '\0';
```



```
250     }
251     taskEXIT_CRITICAL();
252
253     if (keyLocal < 32 || keyLocal > 126) keyLocal = '-';
254
255     // teleplot serial logic
256     if (ENABLEPLOTTER) {
257         if (validLocal) {
258             const int16_t dist_x10 = (int16_t)(dLocal * 10u)
259             ;
260             const int16_t err_x10  = (int16_t)(errLocal *
261             10);
262             // output relative to SERVO_POS_MIN: 0 = closed,
263             positive = opening
264             const int16_t out = (int16_t)((int16_t)outLocal
265             - (int16_t)SERVO_POS_MIN);
266
267             if (!smoothReady) {
268                 smoothDist_x10 = dist_x10;
269                 smoothErr_x10  = err_x10;
270                 smoothOut      = out;
271                 smoothReady    = true;
272             } else {
273                 smoothDist_x10 = emaStep(smoothDist_x10,
274                 dist_x10);
275                 smoothErr_x10  = emaStep(smoothErr_x10,
276                 err_x10);
277                 smoothOut      = emaStep(smoothOut,
278                 out);
279             }
280
281             printf_P(PLOT_DIST,  smoothDist_x10);
282             printf_P(PLOT_SETPOINT, tLocal);
283             printf_P(PLOT_ERROR,  smoothErr_x10);
284             printf_P(PLOT_OUTPUT,  (int)smoothOut);
285             printf_P(PLOT_VALID,   1u);
286
287         } else {
```



```

282         smoothReady = false;
283         printf_P (PLOT_SETPPOINT, tLocal);
284         printf_P (PLOT_VALID, 0u);
285     }
286
287     vTaskDelay (pdMS_TO_TICKS (TASK_PLOT_PERIOD));
288     continue;
289 }
290
291 // text display logic
292 // '=' closed (at or near MIN), '>' opening above MIN +
    deadband
293 // '<' removed - servo never goes below SERVO_POS_MIN
294 char dirChar = '=';
295 if (angleLocal > (uint8_t) (SERVO_ANGLE_MIN +
    SERVO_ANGLE_DEADBAND)) {
296     dirChar = '>';
297 }
298
299 if (validLocal) {
300     printf_P (STR_LINE0, dLocal, tLocal, dirChar);
301 } else {
302     printf_P (STR_LINE0_INVALID, tLocal, dirChar);
303 }
304
305 printf_P (STR_LINE1, keyLocal, localInput);
306 printf_P (PSTR ("\n\n"));
307
308 vTaskDelay (pdMS_TO_TICKS (TASK_DISPLAY_PERIOD));
309 }
310 }

```

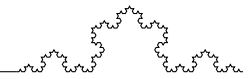
Listing 4: tasks.cpp - Task Implementation

main.cpp

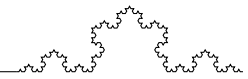
```

1 #include <Arduino.h>
2 #include <Arduino_FreeRTOS.h>
3 #include <Keypad.h>
4 #include <Wire.h>
5 #include <LiquidCrystal_I2C.h>

```



```
6 #include <lcd/LcdStdioManager.h>
7 #include <keypad/KeypadStdioManager.h>
8 #include "actuator.h"
9 #include <serialio/serialio.h>
10 #include <sensors/distance.h>
11 #include "tasks.h"
12
13 // LCD (I2C address 0x27, 16x2)
14 static LiquidCrystal_I2C lcd(0x27, 16, 2);
15
16 // Keypad pin configuration
17 static byte rowPins[4] = {9, 8, 7, 6};
18 static byte colPins[4] = {5, 4, 3, A1};
19
20 // Keypad layout (PROGMEM via Keypad library)
21 static const char keys[4][4] = {
22     {'1', '2', '3', 'A'},
23     {'4', '5', '6', 'B'},
24     {'7', '8', '9', 'C'},
25     {'*', '0', '#', 'D'}
26 };
27
28 static Keypad keypad(makeKeymap(keys), rowPins, colPins, 4, 4);
29
30 // Servo actuator on pin 11
31 static ServoActuator actuator(11);
32 // Ultrasonic sensor from nano simulation: TRIG=A0, ECHO=D2
33 static DistanceSensor distanceSensor(A0, 2);
34 static ControlContext controlContext = {&actuator, &
    distanceSensor};
35
36 void setup() {
37     Serial.begin(9600);
38     actuator.begin();
39     distanceSensor.init();
40
41     lcd.init();
42     lcd.backlight();
```



```
43
44 // STDIO: LCD output, Keypad input
45 static LcdStdioManager lcdManager;
46 // lcdManager.setup(&lcd);
47 KeypadStdioManager::setup(&keypad);
48
49 redirectSerialToStdio(false, true, true);
50
51 // FreeRTOS tasks (stack sizes tuned for Nano - 2KB RAM)
52 xTaskCreate(TaskRead, "Read", 100, NULL, 1, NULL);
53 xTaskCreate(TaskFilter, "Filter", 160, &controlContext, 2,
54            NULL);
55 xTaskCreate(TaskDisplay, "Disp", 140, NULL, 1, NULL);
56
57 // fprintf_P(stderr, PSTR("Ready\n"));
58 vTaskStartScheduler();
59
60 void loop() {}
```

Listing 5: main.cpp - System Initialization